MERN Stack Project Setup Guide

A clear terminal walkthrough with command explanations for VS Code and Command Prompt

1. Architecture & What MERN Stands For

M • MongoDB	E • Express.js	R • React	N • Node.js
Database storage	Backend API routing	Frontend client UI	JavaScript server runtime

2. Command Reference & Why We Write Them

Terminal Command	Full Meaning	What It Actually Does
`mkdir`	Make Directory	Creates a brand new folder on your computer.
`cd`	Change Directory	Moves your terminal context into that specific folder.
`cd ..`	Change Directory (Up)	Moves you back out one level to the parent folder.
`npm init -y`	Node Init (-Yes default)	Generates a `package.json` without asking setup questions.
`npm install`	Install Packages	Downloads libraries from the registry into a `node_modules` folder.

3. Root Setup & Project Directory

Step 1: Create your main workspace

```
mkdir my-mern-app
cd my-mern-app
```

- **Why mkdir?** We create an isolated parent folder so client and server remain organized side-by-side.
- **Why cd?** Your terminal still points to where you were; `cd` steps you directly inside the new project.

4. Backend Server Setup (Node.js & Express)

Step 2: Initialize Node and install server packages

```
mkdir server
cd server
npm init -y
npm install express mongoose cors dotenv
```

- **express:** The web framework that handles URL routes and incoming HTTP requests.
- **mongoose:** Simplifies reading and writing data to your MongoDB database.
- **cors:** Cross-Origin Resource Sharing. Allows React (port 5173/3000) to communicate with Node (port 5000).
- **dotenv:** Securely loads secret credentials (passwords, database URLs) from a `.env` file.

4. Backend Continued: Minimal Server File

Step 3: Create index.js inside your server folder

```
// In Windows Command Prompt:
type nul > index.js
// In VS Code Terminal / Git Bash:
touch index.js
```

- **index.js** is the entry point where Express starts, connects to MongoDB, and listens on a port (e.g., port 5000).
- In your `server/package.json`, add `"type": "module"` if you want to use modern `import` statements instead of `require()`.
- To return to the project root for the next step, run: `cd ..`

5. Frontend Setup (React via Vite)

Step 4: Scaffold client application

```
npm create vite@latest client -- --template react
cd client
npm install
npm install axios
```

- **npm create vite@latest:** Quickly creates a clean, lightweight React app without the heavy baggage of older tools.
- **cd client && npm install:** Steps into the newly built UI folder and downloads the base React packages.
- **axios:** A promise-based HTTP client used inside React components to send GET/POST requests to your Express API.

6. Resulting Project Structure

```
my-mern-app/
├── server/              (Backend)
│   ├── node_modules/
│   ├── index.js          Express server code
│   ├── .env              MongoDB connection string
│   └── package.json
└── client/              (Frontend)
    ├── src/              React components (App.jsx, main.jsx)
    ├── package.json
    └── vite.config.js
```

7. Daily Running Workflow in VS Code

Running both apps together

- Open **two split terminals** in VS Code (`Ctrl + Shift + ` ` or click the split icon).
- **Terminal 1 (Backend):** Type `cd server`, then run `node index.js` (or `npx nodemon index.js` to auto-restart on changes).